Ph.D. Proposal. (Gayani Gupta)

Scheduling Data-Intensive DAG Workflows Across Federated Fog Systems: From Static Placement to Dynamic Partial Repartitioning

Gayani Gupta
Department of Computer Science and Engineering
University of North Texas
Advisors: Dr. Mohsen Amini Salehi and Dr. Sanjukta Bhowmick

October 12, 2026

Abstract: Time-critical workflows—flood forecasting, wildfire tracking, evacuation planning, offshore platform monitoring—run as directed acyclic graphs (DAGs) of dependent tasks. These workflows must produce answers under hard deadlines, exactly when the wide-area link to the cloud is degraded or unavailable, so the computation has to happen on fog resources federated around the event. Scheduling in this setting is harder than in the cloud: every edge of the DAG carries data that may have to cross a slow link, and every task can trade result accuracy for execution time. Existing schedulers treat these aspects in isolation—or ignore them entirely—and almost all of them assume the schedule is computed once, before execution, and never revisited.

This research develops a three-part treatment of the problem. First, we formulated the joint allocation problem—task placement, per-edge data movement, per-link bandwidth, and per-task approximate execution under a global deadline—as a mixed-integer program with correctness proofs, and built three scalable schedulers to solve it (published at IEEE Cloud Summit 2025). Second, we hardened that foundation into a data-aware, measurable evaluation: a maximum-accuracy bound as a reference for every method, a retry-once failure model separating accuracy from feasibility, a bandwidth-degradation study exposing what bandwidth-blind schedulers get wrong, and honest scalability limits for each solver. Third, we propose to remove the static assumption itself: when a whole fog resource fails mid-workflow, the goal is to identify the smallest affected region of the task DAG, repartition and reassign only that region onto surviving resources, update only the communication paths that changed, and resume execution without restarting completed work—recovering faster with less disturbance than a complete repartition.

1 Introduction

Workflows that respond to disasters, such as flood forecasting, wildfire tracking, and evacuation planning, run as directed acyclic graphs (DAGs) of dependent tasks [4, 5]. A flood-response workflow, for example, chains sensing, hydrological modeling, forecast computation, evacuation planning, and alert dissemination. The deadline is part of the specification: a forecast that arrives after the water has risen is a forecast nobody uses.

These workloads cannot rely on the cloud. The same event that creates the deadline—the flood, the fire, the platform incident—degrades the wide-area network. The practical answer is the compute continuum: edge devices acquire data, cloud and HPC provide elastic capacity, and between them

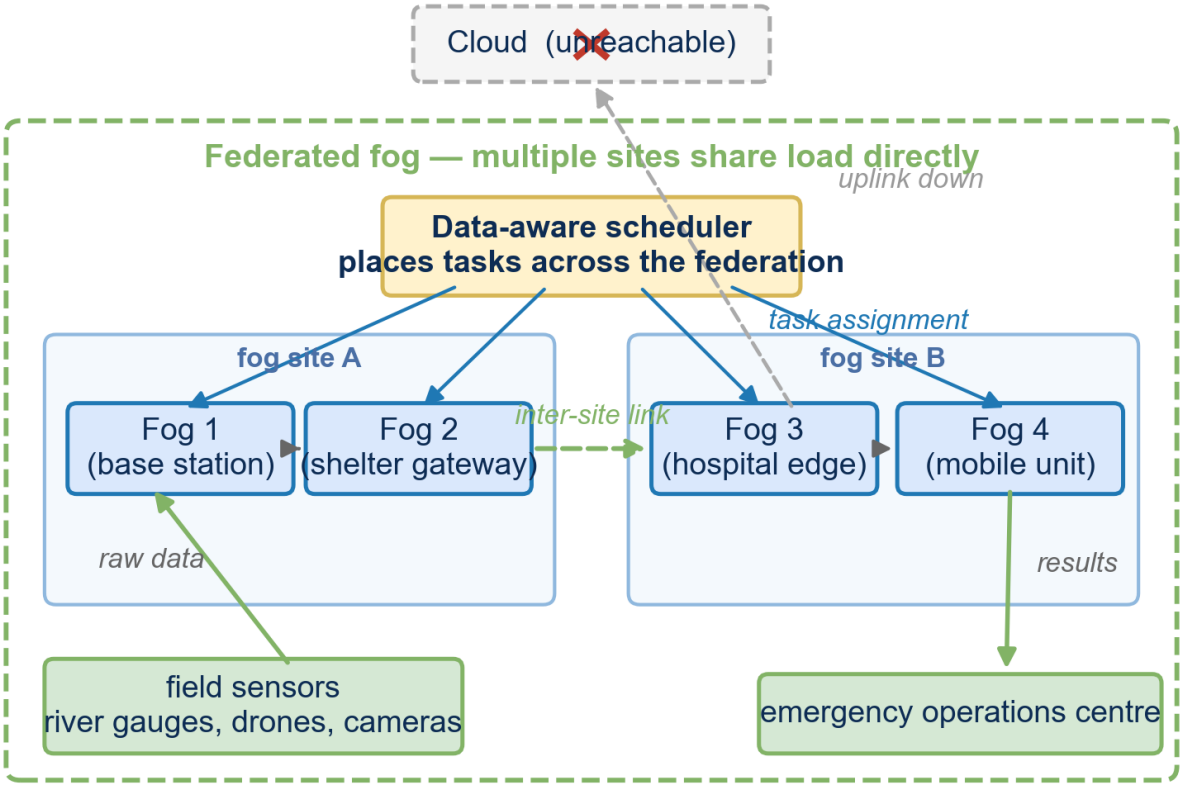


Figure 1: The deployment setting. Sensors feed a response workflow that must execute on fog sites federated around the disaster area, because the uplink to the cloud is degraded or lost precisely when the workflow is needed.

sit fog resources—cell-tower racks, roadside cabinets, platform servers—close enough to the event to matter [6, 7, 8, 9]. Individually a fog site is too weak for the whole workflow, but nearby sites keep fast, stable links to each other even when the cloud link dies; federating them creates a small local cloud. The price is a placement decision for every task, and every placement decision ships data across a link.

What makes this problem genuinely hard are two properties that interact. First, data movement rivals computation: every DAG edge carries an intermediate data volume D_{ji}, and when the producing task j and consuming task i run on different resources, that data crosses a link of bandwidth B_{ba} at cost D_{ji}/B_{ba}—comparable to a task's own execution time. A "fast" remote node can lose more time receiving its input than it saves executing. Second, accuracy is a tradeable resource: tasks in these workflows are approximate—running a fraction ℓ of the work costs ℓ of the time and returns ℓ of the accuracy. A flood model at 80% fidelity still warns in time. The two choices interact: placement decides the transfers, fractions decide the runtimes, and both compete for the same deadline.

All of this still assumes the schedule is computed once, before execution. In a disaster zone that assumption fails hardest: power loss, a severed link, or hardware failure can kill a whole fog resource mid-workflow, stranding every unfinished task assigned to it. Recomputing the complete schedule is too slow, moves tasks that were running correctly, discards completed work, and adds migration

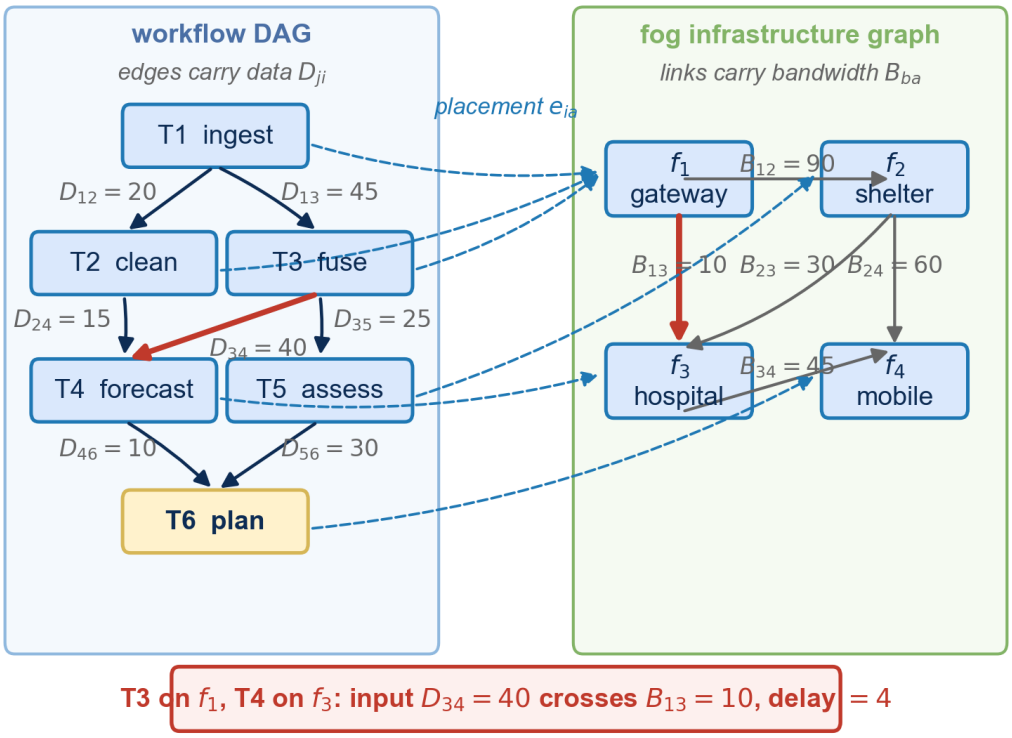


Figure 2: The scheduling problem at a glance: the task DAG (left) is mapped onto the infrastructure graph (right). Placing T_4 remotely makes edge (T_3, T_4) pay D_{34}/B across the link; the same transfer is free if the tasks are co-located.

traffic the degraded network cannot afford. The sensible move is to repair only what broke.

The work therefore falls into three objectives, one per stage of the research:

a) Formulate the joint data-aware scheduling problem for DAG workflows on federated fog—placement, per-edge data movement, per-link bandwidth, and approximate execution under a deadline—and build schedulers to solve it. (Completed; published at IEEE Cloud Summit 2025 [1].)

b) Make the model and its evaluation honest and measurable: an accuracy bound as a reference, a retry-once failure model, a bandwidth-degradation study, and quantified scalability limits. (Completed; submitted [2, 3].)

c) Design a dynamic partial-repartitioning method that repairs only the affected region of a workflow after a resource-level failure. (This proposal.)

Summary of proposed solutions:

a) Joint allocation model + four schedulers. A mixed-integer linear program couples placement, data movement, and execution fractions under a deadline, with correctness proofs; three scalable methods—LP relaxation with randomized rounding, a data-aware greedy, and a genetic algorithm—share the same cost model.

b) A measurable, stress-tested platform. A maximum-accuracy bound lets every method be scored against a reference; a retry-once failure model separates accuracy from deadline feasibility; a bandwidth-degradation sweep with a bandwidth-blind baseline shows that a scheduler can report a perfect score while producing a late schedule; and each solver's breaking point is measured rather than assumed.

c) Minimal-region repair. On resource failure, a boundary rule grows the smallest affected subgraph from the stranded tasks; a region solver repartitions only that subgraph under a recovery cost model; incremental shortest-path updates restore communication paths for moved tasks; and the workflow resumes without restarting completed or unaffected work.

The rest of the document follows that order. Section 2 reviews related work, structured around the three objectives. Section 3 describes the methodology for each of them. Section 4 presents preliminary results and publications. Section 5 covers what remains and the milestones. Section 6 concludes.

2 Background and Related Works

This section reviews the literature relevant to the three objectives in turn.

2.1 DAG task scheduling on heterogeneous and federated systems

The reference point for DAG scheduling on heterogeneous machines is HEFT [11], which ranks tasks by upward rank and places each where it finishes earliest. CPOP appears in the same study, and PEFT [12] refines the template with an optimistic cost table. These list schedulers share three assumptions that break in our setting: transfers appear only as static edge weights, tasks always execute completely, and the plan is fixed at submission. Workflow scheduling on grids and clouds is surveyed by Yu and Buyya [4]; deadline-constrained variants such as Abrishami et al. [13] partition the workflow and distribute the deadline across partitions, but again assume atomic tasks and stable resources. Characterization work by Juve et al. [5] provides the structural profiles of real scientific workflows—Montage, CyberShake, LIGO—that motivate our layered-DAG generator.

In the federated-fog tier, schedulers must additionally respect eligibility (not every task can run everywhere), limited capacity, and the possibility that the cloud path is unusable. Our first objective [1] builds in this line: it keeps the list-scheduling template but prices each transfer by the actual link bandwidth between the chosen nodes, rather than a static edge weight.

2.2 Data-aware and approximate scheduling

Fog and edge schedulers manage capacity, eligibility, and heterogeneity across the compute continuum [9, 10]. Placement policies increasingly price data locality, but rarely model per-link bandwidth explicitly, treat tasks as atomic, and decide the schedule up front.

Approximate computing, surveyed by Mittal [14], trades result quality for time or energy via loop truncation, reduced precision, or early termination—making accuracy a schedulable resource, exactly what tight deadlines need. However, that literature assumes data is already where it needs to be; inter-node transfer cost is not part of the trade. Our second objective [2] joins the two: per-task execution fractions priced against per-edge data volumes and per-link bandwidth, measured

Table 1: Capability comparison: the gap this proposal fills.

Capability	HEFT/classic	Fog sched.	Obj. (a)+(b)	Obj. (c)
Placement on heterogeneous nodes	yes	yes	yes	yes
Per-edge data + per-link bandwidth	no	partial	yes	yes
Approximate execution fractions	no	no	yes	yes
Failure recovery	restart	reschedule all	retry task	repair region

against an explicit accuracy bound and a bandwidth-blind baseline that exposes what ignoring the links costs.

2.3 Failure recovery, rescheduling, and dynamic graph updates

The closest literature to the third objective is recovery-oriented. Sakellariou and Zhao [15] reconsider parts of a workflow at carefully chosen rescheduling points, but remap the workflow globally at each point. Yang et al. [16] reschedule in fog systems under uncertain events; Lee et al. [17] study task redistribution when fog resources join and leave. Classic fault tolerance—checkpoint/restart and replication—recovers lost work at recompute or replication cost but never minimizes the repair region.

On the graph side, Riazi et al. [18] and Khanda et al. [19] update a single-source shortest-path tree after a graph change, touching only the paths affected by the update, and Srinivasan et al. [20] apply the same incremental philosophy to strongly connected components. None of this machinery has been applied to workflow repartitioning.

Table 1 summarizes the gap: every capability exists in isolation, but no work couples an infrastructure-node failure to the smallest affected region of the task DAG.

3 Research Methodology

3.1 Objective (a): joint data-aware allocation — completed

Model. The workflow is a task DAG $G_T = (T, E)$; task T_i has execution time t_{ia} on resource f_a, full-execution accuracy A_i, an eligibility set $P_i \subseteq R$, and an execution fraction $\ell_i \in [0, 1]$ costing ℓ_i of the time and returning ℓ_i of the accuracy. The infrastructure is a resource graph $G_R = (R, L)$ whose links (b, a) carry bandwidth B_{ba}. The communication delay of edge (j, i) is

$$c_{ji}^{ba} = \begin{cases} D_{ji}/B_{ba} & \text{if } b \neq a, \\ 0 & \text{if } b = a \text{ (co-located).} \end{cases} \tag{1}$$

The problem—place each task on one eligible node, choose its fraction, respect precedence-plus-data-arrival order, finish by $t_{\max}$, maximize mean accuracy—is NP-hard even on unrelated parallel machines.

Solvers. Four schedulers share this cost model so that differences are algorithmic, not accounting (Figure 3): (i) an exact MILP with a big-M linearization of the product terms and a five-lemma correctness proof; (ii) an LP relaxation with randomized rounding—relax assignments to $[0, 1]$, sample each task's node, re-solve for fractions; (iii) a data-aware greedy with priority $\beta A_i + (1 - \beta)\, CP_i$ and earliest-finish placement, which on a deadline miss retries with a lower global fraction;

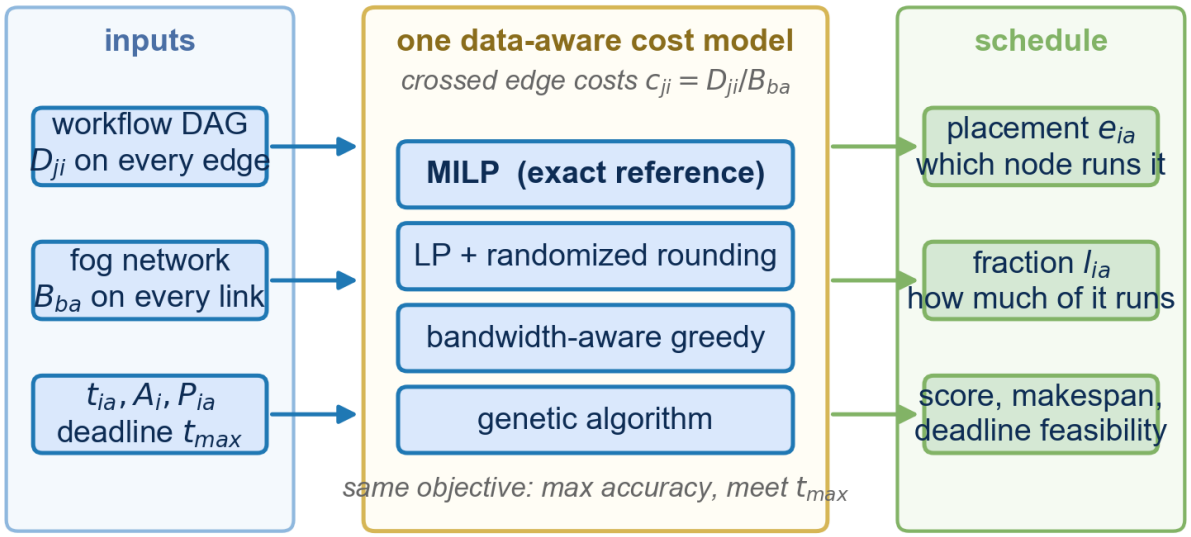


Figure 3: One cost model, four schedulers: the same inputs drive an exact MILP and three scalable methods, producing a placement plus per-task execution fractions evaluated against the bound.

and (iv) a genetic algorithm whose chromosome encodes assignment and fractions together, with a co-location repair for heavy edges.

3.2 Objective (b): a measurable, stress-tested platform — completed

The second objective hardened the foundation in four ways. (i) Bound. A maximum-accuracy bound—the score achievable if every task ran fully on its best eligible node with instantaneous data arrival—gives every method a reference at every operating point. (ii) Failure semantics. A retry-once model reruns a failed task in place and keeps its accuracy only if the retry still meets the deadline, which cleanly separates accuracy from feasibility. (iii) Bandwidth degradation. Scaling every link from 1.0× down to 0.05× nominal, with a bandwidth-blind baseline that ignores transfer cost, isolates exactly what data-awareness buys. (iv) Scalability limits. Each solver's breaking point is measured: the MILP near 800 assignment variables, the LP near 10^5 constraint rows at 1,000 tasks, and the GA under a fixed search budget. Structural graph generation is verified to 10^6 vertices.

3.3 Objective (c): dynamic partial repartitioning — proposed

The failure event. At time τ, resource $r^* \in R$ fails and is removed from G_R. The task set partitions into T^{done}, T^{run}, and T^{pend}; the stranded set is $S = \pi^{-1}(r^*) \cap (T^{\text{run}} \cup T^{\text{pend}})$. Two invariants govern everything that follows: tasks in T^{done} never restart, and assignments of unaffected tasks

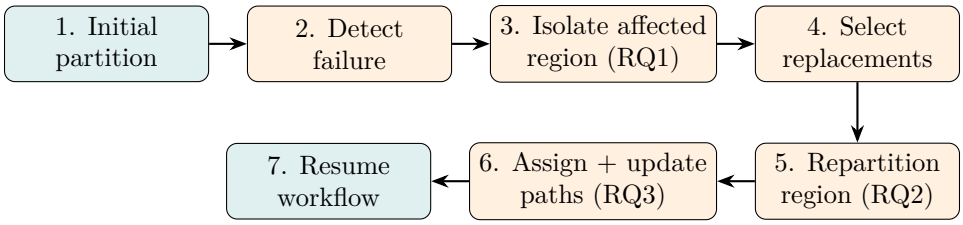


Figure 4: The recovery pipeline. Teal stages reuse machinery that already exists (the Stage-1 schedulers and the execution platform); orange stages are the proposed work, each mapped to a research question.

never move.

Formal problem. Given G_T, the surviving infrastructure $G_R \setminus \{r^*\}$, the original schedule π, the task states at τ, and $t_{\max}$: find an affected region $A \supseteq S$ and a repair assignment $\pi' : A \to R \setminus \{r^*\}$ minimizing the recovery cost

$$C_{\text{rec}} = \underbrace{\Delta_{\text{makespan}}}_{\text{added finish time}} + w_1 \cdot \underbrace{|\pi' \neq \pi|}_{\text{tasks moved}} + w_2 \cdot \underbrace{\text{bytes re-shipped}}_{\text{data movement}}, \tag{2}$$

subject to the invariants above, deadline feasibility, and minimal accuracy loss. Complete repartition is the special case $A = T$; the claim to test is that for realistic failures $|A| \ll |T|$.

Stage 1 — Initial partition. Partition G_T and assign groups to resources under execution time, capacity, communication cost, and deadline, reusing the schedulers of objectives (a) and (b).

Stage 2 — Failure detection. Monitoring marks r^* failed; the system identifies the stranded set S.

Stage 3 — Boundary rule and affected-subgraph extraction (RQ1). Construct A by expansion from S: repeatedly admit a neighbor task $v \notin A$ while v fails a stay-test:

$$\text{deadline:} \quad \hat{y}_v(\pi) \;>\; \tau + s_v(\tau), \tag{3}$$

$$\text{locality:} \quad \min_{P \in \text{paths}(u \to v)} D_{uv}/B_P \;>\; \delta_v, \tag{4}$$

where $\hat{y}_v(\pi)$ is v's finish time under the original schedule, $s_v(\tau)$ the slack remaining at τ, and δ_v a per-task locality budget derived from its successors' start times. Expansion stops when no neighbor fails a test or $|A|$ reaches a cap bounding repair effort. The output is the affected subgraph plus its boundary pins—stationary tasks whose data-arrival times constrain the repair.

Stage 4 — Replacement-resource selection. From $R \setminus \{r^*\}$ select nodes with the capacity and memory for A, ranked by shortest-path distance to the resources hosting A's parents and children.

Stage 5 — Repartition the region (RQ2). Repair is a smaller instance of the same optimization family—deliberately, so the Stage-1 schedulers become region solvers (greedy for millisecond-scale repair, MILP as reference on small regions):

$$\min_{\pi' :\, A \to R \setminus \{r^*\}} \quad C_{\text{rec}}(\pi') \tag{5}$$

$$\begin{aligned}
\text{s.t.} \quad & \pi'(v) \in P_v \cap \big(R \setminus \{r^*\}\big) && \forall\, v \in A, \\
& y_j + c_{ji}^{\pi'(j)\,\pi'(v)} \;\leq\; M_v && \forall\, (j,v) \in E \cap A, \\
& a_{u \to v} \text{ fixed} && \forall\, (u,v) :\; u \notin A,\; v \in A, \\
& y_v \;\leq\; t_{\max} && \forall\, v \in A.
\end{aligned}$$

The third constraint is the boundary-pin condition: arrival times from stationary predecessors are constants, so the region sees the rest of the DAG as a fixed interface.

Stage 6 — Assignment and incremental path update (RQ3). Assign new partitions, then update communication paths between moved and stationary tasks incrementally, adapting the dynamic single-source shortest-path updates of [18, 19]. Recomputation per source costs $O(|L|+|R|\log|R|)$; the incremental update touches only the $\Theta(|A|)$ moved tasks and the paths through r^*, so recovery cost scales with the damage, not the infrastructure. This is a novel application of dynamic-graph machinery.

Stage 7 — Resume. Execution resumes from the last successful point: completed outputs are kept, unaffected tasks run undisturbed, repaired tasks start when their inputs arrive.

4 Preliminary Results

4.1 Objective (a) — published

The Cloud Summit study [1] established the formulation and compared the four schedulers across deadline slack, federation size, workflow size, and task-failure rate: the MILP is the quality reference; relaxation and the GA track it most closely; the data-aware greedy is the low-overhead option under pressure.

4.2 Objective (b) — completed, under submission

The data-aware study [2] answers five questions, each isolated by a sweep over five fresh instances per point:

- Slack ($\alpha \in [1.0, 2.0]$): at $\alpha = 1.0$ the MILP leads (1.38 vs. bound 1.43); from $\alpha = 1.2$ the greedy reaches the bound. Method choice matters most exactly where the deadline binds (Figure 5).
- Federation ($N \in \{2, \ldots, 10\}$): MILP and greedy match the bound at every size; GA and rounding trail up to $\sim$8%. Capacity buys feasibility, not accuracy.
- Scale ($n \in \{10, 100, 1000\}$): at $n = 1000$ the greedy holds the bound in 0.15 s; rounding costs $\sim$93 s per LP re-solve; the GA falls to 1.10 in $\sim$159 s.
- Failures (rate 0–0.5, retry-once): scores stay flat but feasibility separates the methods—greedy 100% vs. rounding $\sim$20% at rate 0.5, because early finishes leave room for retries.
- Bandwidth ($\times 1.0 \to 0.05$): the bandwidth-blind baseline reports bound-level scores at every level while its schedules turn infeasible (80% at 0.5×, 0% below 0.2×); data-aware methods trade accuracy for feasibility and degrade gracefully (Figure 6).

The platform is reproducible end-to-end—code, CSVs, executed notebook, figures, and slides are public [3]—and its graph generators produce million-vertex task DAGs and resource graphs in seconds, the scales the failure-injection experiments of objective (c) will need.

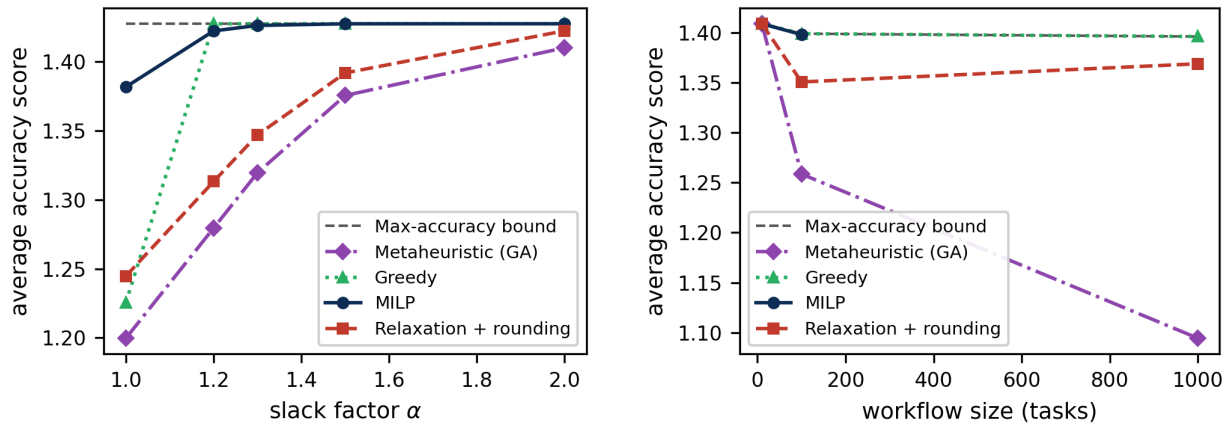


Figure 5: Mean accuracy vs. the max-accuracy bound (dashed): left, deadline-slack sweep; right, workflow-size sweep.

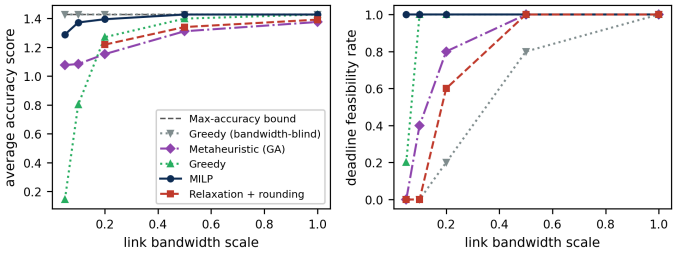


Figure 6: Bandwidth-degradation sweep: the blind baseline (dashed, bound-level scores) produces increasingly late schedules as links degrade.

4.3 Publications

I. G. Gupta, A. Gupta, M. Amini Salehi, A. Fernández Anta, S. Bhowmick, J. Aguilar, and M. Kourouma, "Towards efficient allocation of tasks in DAG-based workflows across federated fog systems," 2025 IEEE Cloud Summit, pp. 124–129, 2025.

II. G. Gupta, A. Gupta, M. Amini Salehi, A. Fernández Anta, S. Bhowmick, and J. Aguilar, "Data-aware scheduling of approximate DAG workflows in federated fog systems," manuscript submitted, 2026.

4.4 Contributed abstracts and talks

I. G. Gupta, "Data-aware scheduling of approximate DAG workflows in federated fog systems," one-page research abstract, Dutch–Belgian Database Day (DBDBD) 2026, Delft.

5 Remaining Work and Milestones

5.1 Objective (b) — camera-ready follow-through

The data-aware study is written, formatted to the target venue (IEEE conference template, double-blind), and under submission. Remaining work is limited to review-driven revisions and, if accepted, the camera-ready pass: restoring authors, acknowledgments, and the repository link.

5.2 Objective (c) — the proposed work

Three research questions decompose the proposal, each mapped to a stage of Figure 4:

RQ1 — Boundary (Stage 3).
Formalize and validate the stay-tests of Eqs. (3)–(4): prove or empirically establish that the region they grow is minimal yet large enough to repair, and measure how the region grows with failure location and remaining slack.

RQ2 — Repair (Stages 4–5).
Implement the region solver for the problem of Eq. (5), from a greedy millisecond-scale repair to MILP repair as the small- region reference.

RQ3 — Paths (Stage 6).
Adapt incremental SPT updates to moved-task paths and measure where they beat full recomputation.

Evaluation will compare against complete repartition, greedy rescue, a no-repair static baseline, and MILP repair on small cases, over failure location (critical path vs. peripheral), failure count, remaining slack, workflow size, and federation size. Three hypotheses: H1 recovery time scales with $|A|$ not $|T|$; H2 tasks moved and bytes re-shipped are far below the baselines at equal feasibility; H3 repair quality stays near the MILP reference on small cases.

5.3 Milestones

Papers	Venue	Deadline	Status
Objective (a): allocation	IEEE Cloud Summit 2025	—	Published
Objective (b): data-aware scheduling	CCGrid 2027	Abstract Nov 24 2026; paper Dec 1 2026	Under submission
DBDBD abstract	DBDBD 2026, Delft	Nov 8 2026	Submitted
Objective (c): partial repartitioning	IPDPS 2028 / CCGrid 2028	Abstract ∼Oct 2027 paper ∼Dec 2027	RQ1–RQ3 in development

6 Conclusion

This research develops a complete treatment of data-intensive DAG scheduling on federated fog, structured as three papers on one arc. The first established the joint model—placement, per-edge data, per-link bandwidth, approximate execution—with an exact solver, three scalable methods,

and proofs. The second made that platform measurable: a bound, failure semantics, bandwidth degradation, and honest scale limits, all public as a reproducible artifact. The third, proposed here, removes the static assumption: when a resource fails mid-workflow, repair the smallest affected region instead of rescheduling the world. Together the three contributions move the field from where should the workflow run to what it costs to keep it running when the infrastructure fails—with the platform, cost model, schedulers, and baselines for that final step already built.

References

[1] G. Gupta, A. Gupta, M. Amini Salehi, A. Fernández Anta, S. Bhowmick, J. Aguilar, and M. Kourouma, "Towards efficient allocation of tasks in DAG-based workflows across federated fog systems," 2025 IEEE Cloud Summit, pp. 124–129, 2025.

[2] G. Gupta, A. Gupta, M. Amini Salehi, A. Fernández Anta, S. Bhowmick, and J. Aguilar, "Data-aware scheduling of approximate DAG workflows in federated fog systems," manuscript submitted, 2026.

[3] G. Gupta, "Data-aware scheduling of approximate DAG workflows in federated fog systems: simulator and experiments," https://github.com/gayanigupta/data-aware-scheduling, 2026.

[4] J. Yu and R. Buyya, "A taxonomy of workflow management systems for grid computing," Journal of Grid Computing, vol. 3, no. 3–4, pp. 171–200, 2005.

[5] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," Future Generation Computer Systems, vol. 29, no. 3, pp. 682–692, 2013.

[6] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," Proc. ACM MCC Workshop on Mobile Cloud Computing, pp. 13–16, 2012.

[7] L. M. Vaquero and L. Rodero-Merino, "Finding your way in the fog: Towards a comprehensive definition of fog computing," ACM SIGCOMM Computer Communication Review, vol. 44, no. 5, pp. 27–32, 2014.

[8] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637–646, 2016.

[9] C. Mouradian, D. Naboulsi, S. Yangui, R. H. Glitho, M. J. Morrow, and P. A. Polakos, "A comprehensive survey on fog computing: State-of-the-art and research challenges," IEEE Communications Surveys & Tutorials, vol. 20, no. 1, pp. 416–464, 2018.

[10] S. Yi, C. Li, and Q. Li, "A survey of fog computing: Concepts, applications and issues," Proc. ACM Mobidata Workshop, pp. 37–42, 2015.

[11] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," IEEE Trans. Parallel Distrib. Syst., vol. 13, no. 3, pp. 260–274, 2002.

[12] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," IEEE Trans. Parallel Distrib. Syst., vol. 25, no. 3, pp. 682–694, 2014.

[13] S. Abrishami, M. Naghibzadeh, and D. H. J. Epema, "Deadline-constrained workflow scheduling algorithms for Infrastructure as a Service clouds," Future Generation Computer Systems, vol. 29, no. 1, pp. 158–169, 2013.

[14] S. Mittal, "A survey of techniques for approximate computing," ACM Computing Surveys, vol. 48, no. 4, pp. 62:1–62:33, 2016.

[15] R. Sakellariou and H. Zhao, "A low-cost rescheduling policy for efficient mapping of workflows on grid systems," Scientific Programming, vol. 12, no. 4, pp. 253–262, 2004.

[16] Y. Yang, F. Ren, and M. Zhang, "A decentralized multiagent-based task scheduling framework for handling uncertain events in fog computing," arXiv:2401.02219, 2024.

[17] G. Lee, W. Saad, and M. Bennis, "An online optimization framework for distributed fog network formation with minimal latency," arXiv:1710.05239, 2019.

[18] S. Riazi, S. Srinivasan, S. K. Das, S. Bhowmick, and B. Norris, "Single-source shortest path tree for big dynamic graphs," 2018 IEEE Int. Conf. on Big Data, pp. 4054–4062, 2018.

[19] A. Khanda, S. Srinivasan, S. Bhowmick, B. Norris, and S. K. Das, "A parallel algorithm template for updating single-source shortest paths in large-scale dynamic networks," IEEE Trans. Parallel Distrib. Syst., vol. 33, no. 4, pp. 929–940, 2021.

[20] S. Srinivasan, A. Khanda, S. Srinivasan, A. Pandey, S. K. Das, S. Bhowmick, and B. Norris, "A distributed algorithm for identifying strongly connected components on incremental graphs," 2023 IEEE 35th Int. Symp. on Computer Architecture and High Performance Computing (SBAC-PAD), pp. 109–118, 2023.